CSE Senior Assignment — Concurrency Forensics: The RideShare Platform

Background

You have just joined a startup as a backend engineer. The previous team shipped a ride-booking platform (in `src/rideshare/`) and left the company. Production is on fire: riders report duplicate charges, drivers complain of being assigned two rides at once, push notifications silently stop arriving, and the payments service occasionally freezes and has to be force-restarted every night.

Management insists the code is fine — "it compiles, it has comments saying it's thread-safe, and it worked in the demo." Your job is to prove otherwise, with evidence.

The System

Six classes, no external dependencies:

File	Role
`RideManager.java`	Singleton coordinator: registration, matching, booking, payments, stats
`Driver.java`, `Rider.java`	Platform actors
`Wallet.java`	Money movement between riders and drivers
`Ride.java`	Ride lifecycle and timestamps
`NotificationService.java`	Async notification dispatch via a background thread
`RideBookingSimulation.java`	A load test with a built-in AUDIT report

Build and run:

```
javac -d out src/rideshare/*.java
java -cp out rideshare.RideBookingSimulation
```

Run it at least ten times. Save the outputs. Notice that the results are different every time — that observation is where this assignment begins.

Part A — Bug Hunt (40 marks)

Identify the concurrency defects in this codebase. There are **at least 10 distinct defects**. For each one you find, document:

1. **Location** — file, method, line numbers.
2. **Classification** — race condition (check-then-act / read-modify-write), deadlock, visibility failure, non-thread-safe class misuse, unsafe publication, liveness/CPU waste, or interruption mishandling.
3. **Evidence** — a specific symptom from your simulation runs (audit FAIL line, log excerpt, thread dump, CPU usage) that this defect explains. A bug report without observed evidence earns at most half marks.
4. **Why the comment lies** — several defects sit under comments confidently claiming thread safety. Explain precisely why the stated justification is wrong (e.g., what "atomic" actually guarantees and what it doesn't).

Hints on evidence gathering: `jstack <pid>` for thread dumps, `jconsole` /VisualVM for live monitoring, and `top -H` for per-thread CPU. The simulation prints its own PID hint when Phase 2 hangs.

Part B — Explain the Weird Numbers (15 marks)

In a typical run the audit reports something like: 30 rides in `allRides`, dozens of duplicate ride IDs, yet `rideCounter == 2`; and the notification dispatcher is *alive* but has delivered nothing, with ~90 messages pending.

Write a precise, Java-Memory-Model-grounded explanation of:

1. How the driver-matching race and the ride-ID race **interact** — why do the very threads that slipped through the matching window also share stale counter values?
2. Why the notification dispatcher can be alive, spinning at 100% CPU, and still never observe the queue becoming non-empty. Your answer should use the terms *happens-before*, *visibility*, and explain what the JIT compiler is allowed to do with the loop.
3. Why `System.exit(0)` is needed at the end of `main`, and what would happen without it.

Part C — Fix It (25 marks)

Produce a corrected version of the codebase. Constraints:

- The audit must print all-OK across 20 consecutive runs, Phase 2 must never hang, and all notifications must be delivered before exit.
- You may not fix things by making the program single-threaded, adding giant sleeps, or wrapping everything in one global lock (a solution that serializes all booking will be tested for throughput).
- For each fix, add a short comment: what guarantee the new construct provides that the old code lacked.

Expected toolbox (use what's appropriate, justify each choice): `AtomicInteger`, `ConcurrentHashMap`, `BlockingQueue`, `volatile`, `synchronized` with correct scope, lock ordering or `tryLock`, `java.time.DateTimeFormatter`, `ExecutorService`, proper `InterruptedException` handling (restore the interrupt status — don't swallow it).

Part D — Design Critique & Redesign Proposal (20 marks)

Concurrency bugs rarely live alone; they grow in badly shaped designs. Write a 1–2 page critique covering at least:

1. Why `RideManager` is a God class, and how its singleton + mutable static state makes the system untestable and race-prone. Propose a decomposition (e.g., `DriverRegistry`, `MatchingService`, `PaymentService`, `RideRepository`) with interfaces and dependency injection.
2. Why locking on a `String` literal is dangerous in a way that locking on a private `final Object` is not.
3. Why `getDrivers()` / `getAllRides()` returning internal mutable collections breaks encapsulation *and* thread safety simultaneously.
4. A redesign of driver matching that is correct **by construction** rather than by locking discipline — e.g., a concurrent pool where "claim a driver" is a single atomic operation. Sketch the API.
5. One paragraph: how would this design change if drivers and riders lived on different servers (i.e., which of your in-process fixes stop working in a distributed system)?

Deliverables

1. `BUG_REPORT.md` — Part A table + Part B explanations, with evidence attached.
2. `src-fixed/` — corrected code (Part C).
3. `REDESIGN.md` — Part D.
4. A 10-minute viva: you will be asked to break your own fixed code, or defend why it cannot be broken.

Academic Integrity Note

You may use any references on the Java Memory Model and `java.util.concurrent`. You must be able to explain every line of your fix in the viva; unexplained fixes score zero regardless of correctness.